



Universidad
Nacional
de Quilmes

Trabajo Practico Final UNQ-Shop

Materia: Objetos 2

Comisión: 1

Integrantes:

Rodríguez Julieta

Email: Julieta4730@hotmail.com

López Micaela Alejandra

Email: micaela.lopezz1999@gmail.com

Introducción

El objetivo de este informe es presentar y justificar las decisiones tomadas al diseñar el sistema de gestión para la empresa de venta online “UNQ-Shop”. El desafío principal del trabajo práctico fue conectar de forma limpia las distintas partes del sistema: las búsquedas de productos en el catálogo, el control de los estados por los que pasa un pedido, el pago con sistemas externos, la creación de reportes en diferentes formatos, etc.

Para resolver esto, nos enfocamos en dividir las responsabilidades del sistema en diferentes clases. Decidimos aplicar patrones de diseño específicos para cada problema, logrando que el comportamiento de cada módulo sea independiente. A continuación, se explican las estrategias utilizadas para resolver los requerimientos técnicos del enunciado.

Decisiones de Diseño y Detalles de Implementación

Modelado del Cliente mediante una Interfaz

Para resolver la integración con los **Medios de Pago**, decidimos modelar a Cliente como una interfaz en lugar de una clase concreta. La razón principal es que los distintos medios de pago requerían datos de orígenes totalmente diferentes.

Al definir los métodos dentro de una interfaz, desacoplamos la lógica del cliente de los medios de pago. Esto fue fundamental para las pruebas unitarias del sistema, ya que nos permitió implementar objetos simulados **mocks** en los tests de manera sencilla y práctica.

Ciclo de Vida: El Pedido Confirmado no está Pago

Al diseñar los estados del pedido utilizando el patrón **State**, establecimos que el pedido permanece en el estado **EstadoDePedidoConfirmado** hasta que se invoca el mensaje **pagar()**. Si el proceso de pago se realiza correctamente a través del **MedioDePago**, el pedido cambia hacia el estado **EstadoPedidoEnPreparacion**.

Validación de Datos en Billetera Virtual utilizando CBU y Alias

En la clase abstracta **MedioDePago** en el primer paso del método general para procesar los cobros, en **BilleteraVirtual**, se pide que haya “saldo suficiente en la cuenta”, por lo cual decidimos que verificar si la cuenta asociada tenía fondos suficientes antes de avanzar al siguiente paso, sea validar los datos del cliente teniendo el Cbu y Alias del mismo.

Patrones de diseño utilizados y roles

Patrón Composite (Catálogo de productos)

Para poder tratar de la misma forma a un producto individual y a un paquete de productos implementamos el patrón **Composite**, en el cual definimos los roles de la siguiente manera según la definición de Gamma et.al.

- **Component (Producto):** Es la clase abstracta que define los métodos comunes para todo el catálogo. Esto hace que se puedan tratar a cualquier producto por igual, ya sea producto individual o paquete.
- **Leaf (ProductoIndividual):** No contiene a otros productos. Implementa los métodos devolviendo sus propios datos.
- **Composite (Paquete):** Es el producto compuesto. Guarda una lista de tipo `Producto`. Sobrescribe los métodos para delegar el trabajo de forma recursiva a sus hijos.
- **Client (Pedido / Sucursal):** Usan los objetos a través de la clase madre `Producto`.

Patrón State (Ciclo de Vida del Pedido)

Utilizamos el patrón **State** para permitir que el pedido altere su comportamiento cuando cambie su estado interno, definimos los roles de la siguiente manera según la definición de Gamma et.al.

- **Context (Pedido):** Tiene la referencia al estado actual y expone los métodos. delegando la ejecución de los mismos en el objeto según en el estado en el que esté.
- **State (EstadoDePedido):** Interfaz que define las acciones que se pueden hacer en el ciclo de vida.
- **Concrete States (EstadoDePedidoBorrador, EstadoDePedidoConfirmado, etc.):** Proporcionan sus propias implementaciones para los métodos específicos de cada estado.

Patrón Strategy (Métodos de Envío)

Como el cálculo de costos y los días de entrega varían según lo que elija el cliente, utilizamos el patrón **Strategy** y definimos los roles de la siguiente manera según la definición de Gamma et.al.

- **Context (Pedido):** Mantiene una referencia al método de envío elegido.
- **Strategy (MetodoDeEnvio):** Interfaz de estrategia común a todas las variantes de algoritmo.
- **Concrete Strategies (EnvioEstandar, EnvioExpress, RetiroEnSucursal):** Cada una implementa sus distintas variaciones del algoritmo que utiliza el Context.

Patrón Template Method y Adapter (Métodos de Pago)

Como el procesamiento de los pagos necesita seguir una serie de pasos fijos y ordenados, implementamos el patrón **Template Method** y el **Adapter** ya que nos proporcionaban una

Template Method

- **AbstractClass (MedioDePago):** Define la estructura fija del proceso en el método `pagar()`. Los pasos específicos son métodos abstractos que implementa cada hijo.
- **ConcreteClass (TarjetaDeCredito, TransferenciaBancaria, BilleteraVirtual):** Implementan esos pasos según cómo funcione cada medio de pago.

Adapter

A su vez, estas clases concretas actúan como **Adaptadores** para poder consumir las APIs externas de los bancos o tarjetas, que tienen firmas de métodos diferentes a las del sistema.

Patrón Observer (Notificaciones del Pedido)

Utilizamos el patrón **Observer** para notificar a los clientes de forma automática los cambios de estado de los pedidos y definimos los roles de la siguiente manera según la definición de Gamma et.al.

- **Subject / Observable (Notificador):** Es el que maneja la lista de los observadores(clientes) y se encarga de avisarles cuando hay un cambio en el estado. Guarda una lista y expone métodos para que se puedan registrar o dar de baja. Cuando el pedido cambia de estado, se encarga de notificar.
- **Observer (Sistema):** Es la interfaz común que tienen que implementar todos los que quieran quedarse enterarse sobre los cambios del pedido. Define el método clave para avisar del cambio de estado.
- **Concrete Observers (Fidelización / GeneradorDeFactura):** Son las clases concretas que reaccionan e implementan su propia lógica cuando les llega el aviso.
- **Client (Pedido):** Es el que tiene la referencia al notificador y, cada vez que cambia su estado actual, le pide que mande la alerta a todos los suscriptores.

Patrón Composite (Búsqueda en el Catálogo)

Para los filtros de búsqueda en el catálogo, utilizamos el patrón **Composite** para poder combinar filtros de forma dinámica, definiendo los roles de la siguiente manera según la definición de Gamma et.al.

- **Component (CriterioDeBusqueda):** Interfaz que define el método clave.

- **Leaf (CriterioPorCategoria, CriterioPorNombre, etc.):** Filtros simples que evalúan una sola propiedad del producto.
- **Composite (CriteriosConjuncion, CriteriosDisyuncion):** Clases que combinan dos criterios usando AND, OR, evaluando de forma recursiva los sub-criterios.
- **Client (Sucursal):** Quien produce la filtración de búsqueda en el catálogo.

Patrón Visitor (Reportes)

Para permitir que un mismo reporte se pueda exportar en múltiples formatos (**HTML**, **CSV**, **Texto Plano**) aplicamos el patrón **Visitor**, definiendo los roles de la siguiente manera según la definición de Gamma et.al.

- **Visitor (FormatoDeExportacionDeReporte):** Es la interfaz abstracta del visitante. Declara una operación de visita para cada clase de elemento concreto en la jerarquía de reportes.
- **Concrete Visitors (Html, Csv, TextoPlano):** Son las clases concretas que implementan cada una de las operaciones declaradas por la interfaz del Visitor.
- **Element (TipoDeReporte):** Es la interfaz de la jerarquía de elementos que define la operación de aceptación.
- **Concrete Element (ReporteProductosMasVendidos):** Implementa la operación de aceptación mediante el mecanismo de double dispatch (doble despacho).

Conclusión

La aplicación de los patrones de diseño mencionados ha permitido construir un sistema altamente desacoplado y adaptable a los cambios requeridos por la lógica de negocio de UNQ-Shop.

Gracias al uso de interfaces y estructuras compuestas, el código no solo cumple con los requerimientos funcionales, sino que facilita futuras expansiones, como la incorporación de nuevos métodos de pago, filtros o formatos de reporte; sin necesidad de modificar el núcleo del sistema existente.